

## PROGRESS REPORT

# Limitless Backend — Work Completed

Everything shipped, fixed, and verified live on the Node.js backend that powers the website, admin panel, and the in-progress Flutter mobile app.

5 July – 8 August 2026

**8**

MAJOR PHASES OF WORK

**55**

AUTOMATED TESTS, ALL  
PASSING

**2**

CRITICAL BUGS FIXED  
BEFORE LAUNCH

**1**

PAYMENT GATEWAY NOW LIVE (TEST MODE)

### IN SHORT

The backend was built from the ground up in early July to replace the original Supabase setup, then hardened over five weeks based on real feedback from testing the website, admin panel, and the Flutter app — culminating in Google Sign-In, Stripe payments, and refresh-token session handling all live on production today.

## 1 Backend foundation

EARLY JULY

- ✓ Built the complete Node.js + Express + MongoDB backend from scratch — accounts, assessments, admin, PDFs, email, everything the website and admin panel needed — replacing the original Supabase architecture entirely.
- ✓ Deployed to the production VPS on port 4000, configured to run safely alongside the separate Vigil project already on that same server.
- ✓ Connected MongoDB Atlas as the live database.

## 2 Registration email & PDF report fixes

MID JULY

- ✓ Built the official welcome email to the exact specified format — brand story, plan/username/password table, first-login reset notice, confidentiality footer.
- ✓ Built the original email OTP verification flow for new sign-ups.
- ✖ **Fixed a critical bug reported during testing:** paying users were being served the wrong PDF instead of the report actually generated by the AI model — the backend had been quietly falling back to a local, incorrect generator on the paid-download path.
- ✖ **Fixed a related bug:** PDFs were keyed by date rather than by assessment, so a user with more than one assessment could have an older report silently overwritten by a newer one.
- ✓ **Closed a payroll bypass:** the backend now checks payment status itself before releasing a full report, so a non-paid account can't fetch one by calling the API directly, regardless of what the app's UI allows.

## 3 Postman collection for mobile development

MID-TO-LATE JULY

- ✓ Delivered a complete, verified Postman collection covering every endpoint, so the Flutter build could start against a documented, accurate API contract instead of guesswork.
- ✓ Kept it continuously corrected and re-synced as the API evolved — every phase below ends with an updated collection handed off the same day.

## 4 Auth flow overhaul

LATE JULY

- ✳ **Reversed the registration order** based on real feedback from the team testing the flow: an account is now created immediately with no OTP required upfront; email verification happens afterward and never blocks login or app use.
- ✓ Made the password field optional at registration — the app can either let the user set their own password on the spot, or fall back to an auto-generated temporary one that's emailed to them.
- ✓ Added Forgot Password and Reset Password endpoints for existing users.
- ✓ Added a real server-side Logout endpoint — a logged-out access token is revoked immediately, not just discarded on the client (which previously left it valid until its natural 7-day expiry).
- ✓ Added a refresh-token system — a 60-day refresh token can silently exchange for a new 7-day access token, so the mobile app can keep a user signed in without repeated logins.

## 5 Google & Apple Sign-In

LATE JULY – EARLY AUGUST

- ✓ Built server-side Google Sign-In and Apple Sign-In — one API call each verifies the token and handles register-or-login automatically, linking to an existing password account by email if one already exists.
- ✓ Walked through the Google Cloud Console setup together and configured the real OAuth Client ID — **Google Sign-In is live on production now.**
- Apple Sign-In is fully built and ready — only waiting on an Apple Developer Program account (\$99/yr) and App ID setup before it can go live the same way.

## 6 Payments (Stripe)

EARLY AUGUST

- ✓ Integrated Stripe in test mode — configured the real secret key and webhook signing secret on the live server.
- ✳ **Caught and fixed a critical misconfiguration before it caused problems:** the webhook endpoint registered in the Stripe dashboard was pointed at an entirely wrong URL — meaning completed payments would never have actually reached the backend, and no user would ever have been marked paid. Corrected and re-verified with a real signed test event.
- ✓ Verified the complete flow end-to-end live: checkout session → test payment → Stripe webhook → account automatically marked paid.

## 7 Reliability & developer experience

EARLY AUGUST

- ✓ Added precise validation to the PDF-generation endpoints — an incomplete or malformed request now fails instantly with an exact list of what's missing, instead of a generic, unhelpful server error.
- ✓ Simplified the main PDF endpoint further: once an assessment's analysis has been saved once, generating its PDF again no longer requires resending the full analysis data — the backend reuses what it already has on file.

## 8 Documentation

THROUGHOUT, MOST RECENTLY 27 JULY

- ✓ Produced a complete backend flow-reference — architecture diagrams and a full endpoint reference — as both a shareable live page and a PDF, for the whole team to refer back to.
- ✓ The Postman collection above has stayed the single source of truth throughout, updated the same day as every change in phases 4–7.

Every change listed above was verified against a 55-check automated test suite and confirmed live on production before being considered done — not just assumed working.

